Projekt R

Ak. god. 2025./2026.

Obrada i analiza slika pomoću neuronskih mreža

Dokumentacija

Članovi:

Bruno Cavor

Luka Čečura

Lucija Kozić

Leona Križanac

Nora Milolović

Vita Pavlović

Pavle Stanarević

Nika Valić

Daniel Žic

Voditelj studentskog tima: Bruno Cavor

Datum predaje: 30.01.2026.

Mentor: Akademik prof. dr. sc. Sven Lončarić

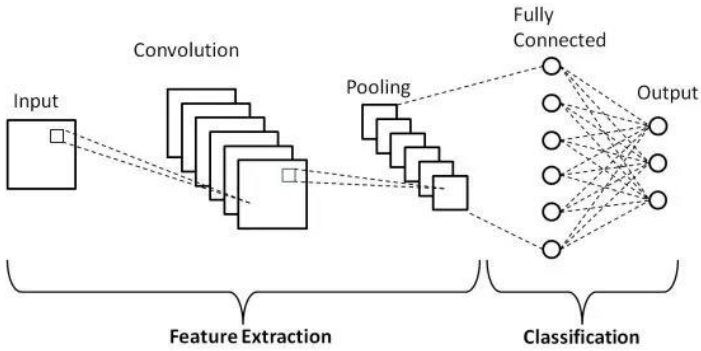
Sadržaj

1 Uvod 2
2 Postupak klasifikacije slika 4
2.1 Prikupljanje i priprema podataka 4
2.2 Ekstrakcija značajki 4
2.3 Treniranje i evaluacija 5
3 Opis korištenih dubokih neuronskih mreža 5
3.1 Mala konvolucijska mreža 6
3.2 ResNet-18 7
3.3 VGG-16 8
3.4 EfficientNetB0 9
3.5 EfficientNet-B1 12
3.6 ConvNeXt 13
4 Opis metoda klasifikacije neuronskih mreža 14
4.1 Treniranje mreže iz nule 14
4.2 Fine-tuning mreže 15
4.3 Treniranje klasifikacijske glave 16
4.4 Linear probe 16
4.5 Stroj s potpornim vektorima (SVM) 17
4.6 k nearest neighbours 18
5 Dobiveni rezultati 19
5.1 Treniranje iz nule 19
5.2 Fine-tuning 20
5.3 Treniranje klasifikacijske glave 23
5.4 Linear probe 25
5.5 SVM 27
5.6 k nearest neighbours 30
6. Zaključak 34
7. Literatura 34

1 Uvod

Umjetna inteligencija je tehnologija koja omogućuje računalima i strojevima da simuliraju kognitivne funkcije čovjeka, kao što su rješavanje problema, donošenje odluka i kreativnost. Današnji strojevi mogu razumjeti govor, rukopis te slike. Grana umjetne inteligencije koja se bavi interpretacijom vizualnih podataka, kao što su slike ili videozapisi, zove se računalni vid. Kako bismo što efikasnije obradili višedimenzionalne podatke poput slika, služimo se konvolucijskim neuronskim mrežama. Konvolucijske neuronske mreže, poznatije kao CNN, su vrsta neuronskih mreža koje koriste trodimenzionalne podatke za klasifikaciju slika i zadatke poput prepoznavanje objekata. Za razliku od neuronskih mreža, koriste konvoluciju umjesto klasičnog množenja matrica u barem jednom njihovom sloju. Sastoji se od više komponenata, kao što su ulazni sloj, konvolucijski sloj, funkcija aktivacije, sloj udruživanja, potpuno povezani sloj i izlazni sloj.

Konvolucijski sloj središnja je komponenta konvolucijske neuronske mreže i odgovoran je za otkrivanje značajki i uzoraka na slikama što postiže primjenom skupa filtera na ulazne podatke.



Slika 1.1: Shematski prikaz konvolucijske neuronske mreže (CNN) koja se sastoji od konvolucijskih i pooling slojeva za ekstrakciju značajki te potpuno povezanih slojeva za konačnu klasifikaciju.
Izvor: Sikander, M., *A Comprehensive Guide to Convolutional Neural Networks*, Medium.

U sklopu projekta implementirali smo klasifikacijske metode bazirane na konvolucijskim neuronskim mrežama. Cilj projekta je upoznati proces samog treniranja konvolucijskih neuronskih mreža kao što su mala konvolucijska mreža s nekoliko slojeva, ResNet-18, VGG-16, EfficientNetB0, EfficientNetB1 i ConvNeXt, razumjeti razlike između različitih načina treniranja te provesti usporedbu metoda na različitim skupovima podataka (Cifar10,

Cifar100 i Imagenette). Klasifikacija slika predstavlja jedan od temeljnih problema računalnog vida, pri čemu je cilj svakoj ulaznoj slici pridružiti odgovarajuću klasu na temelju vizualnog sadržaja. U suvremenim pristupima taj se problem najčešće rješava primjenom dubokih neuronskih mreža, osobito konvolucijskih neuronskih mreža (CNN), koje su se pokazale iznimno uspješnima u ekstrakciji prostornih značajki iz slika.

2 Postupak klasifikacije slika

2.1 Prikupljanje i priprema podataka

U fazi predobrade podataka implementiramo metode kako bismo poboljšali kvalitetu podataka i pripremili ih za sljedeće korake. Skup podataka se dijeli na skup za treniranje (train) koji se koristi za učenje parametara modela, skup za validaciju (val) koji služi za praćenje performansi tijekom treniranja i skup za testiranje (test) koji se koristi kako bi se procijenila sposobnost generalizacije modela. Prije samog treniranja provodi se predobrada podataka. Podatke možemo rezati, normalizirati ih te augmentirati kako bismo poboljšali kvalitetu podataka i poboljšali performanse samog modela.

Rezanjem slika postižemo da nam samo bitni i potrebni podatci ostanu, a sve nepotrebne i nevažne dijelove izrežemo iz slike kako ne bi utjecali na performanse modela. Normalizacija podataka uključuje podešavanje vrijednosti piksela slika na standardnu distribuciju kako bi podaci bili prikladniji za analizu. Augmentacija podataka je proces stvaranja nekih novih varijacija postojećih podataka pomoću rotacije, uvećavanja, okretanja slika, promjene svjetline i kontrasta. Cilj augmentacije je da nam skup za treniranje podataka bude što raznovrsniji i brojniji kako bi naš model bolje generalizirao i bio točniji u klasifikaciji.

2.2 Ekstrakcija značajki

Nakon prikupljanja i pripreme podataka slijedi ekstrakcija značajki. Računalo izdvaja informacije sa slike i određuje pripadnost pojedinih točaka slike određenim značajkama. Cilj ovog koraka je opisati sliku pomoću značajki koje nose bitne informacije potrebne za klasifikaciju. Jednom kada je računalo naučilo te važne značajke i prepoznaje ih u skupu za treniranje, može ih koristiti kako bi klasificiralo nove podatke s kojima nikad prije nije došao

u doticaj. Ekstrakcija značajki poboljšava performanse strojeva jer se fokusira na najbitnije aspekte podataka. Bez ekstrakcije modeli bi trebali analizirati cijelu sliku što bi bilo iznimno sporo i naporno za računalo.

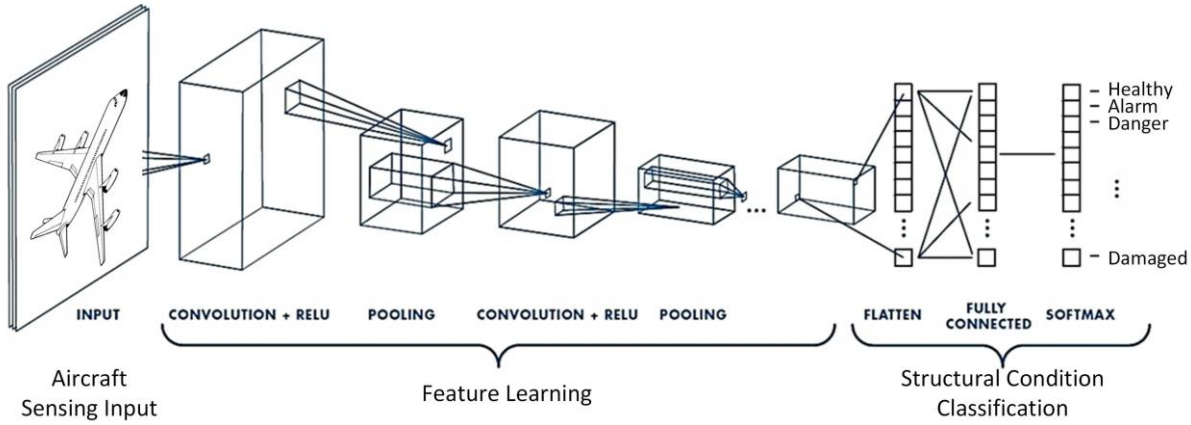
2.3 Treniranje i evaluacija

U ovome koraku koristi se skup za treniranje kako bi model naučio kako svaka klasa izgleda. U tu svrhu primjenjuje se konvolucijska neuronska mreža koja se sastoji od slojeva navedenih u uvodu. Tijekom treniranja težine neuronske mreže, koje određuju jačinu utjecaja ulaznih značajki na izlaz pojedinih neurona, inicijaliziraju se nasumično. Bias parametri koji omogućuju dodatni pomak izlazne vrijednosti neurona, su konstantni. Proces optimizacije temelji se na funkciji gubitka (loss function) i algoritmu povratnog širenja pogreške (error backpropagation). Funkcija gubitka služi kao mjera pogreške klasifikacije, a njezina vrijednost koristi se za izračun gradijenta, što omogućuje postupno poboljšanje same klasifikacije. U zadnjoj fazi, model se primjenjuje na podatke koji još nisu iskorišteni tijekom treniranja i uspješnost klasifikacije se procjenjuje usporedbom stvarnih s predviđenim oznakama.

3 Opis korištenih dubokih neuronskih mreža

3.1 Mala konvolucijska mreža

Mala konvolucijska mreža sastoji se od ulaznog sloja, jednog ili više konvolucijskih slojeva, slojeva sažimanja (pooling) te završnog povezanog sloja. Konvolucijski slojevi primjenjuju filtere nad ulaznim podacima kako bi se izdvojile bitne značajke slike, poput rubova, oblika ili tekstura. Nakon konvolucije, koristi se nelinearna aktivacijska funkcija, najčešće ReLu, koja omogućuje učenje složenijih obrazaca. Pooling slojevi koriste se kako bi smanjili dimenzije podataka i smanjili opterećenje računala te rizik od prenaučenosti (overfitting). Potpuno povezani sloj na kraju mreže koristi izdvojene značajke kako bi donio konačnu odluku npr. klasifikacije slike.



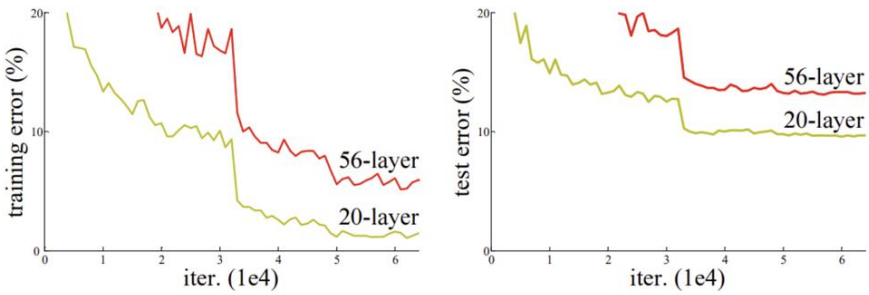
Slika 3.1: Shematski prikaz arhitekture CNN, prikaz slojeva od kojih se sastoji konvolucijska mreža. Izvor: Tabian I, Fu H, Sharif Khodaei Z. A Convolutional Neural Network for Impact Detection and Characterization of Complex Composite Structures. Sensors. 2019;

3.2 ResNet-18

Prilikom korištenja dubokih neuronskih mreža, dodavanjem dodatnih slojeva uočeno je da pogreška treniranja i pogreška testiranja počinju rasti te da se pojavljuju sljedeće anomalije:

a) problem nestajućeg gradijenta (vanishing gradient)

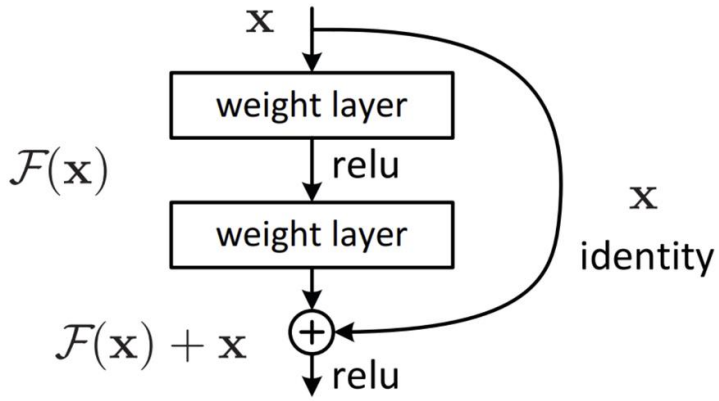
b) problem eksplodirajućeg gradijenta (exploding gradient)



Slika 3.2: Training error (lijevo) i test error (desno) na skupu CIFAR-10 za obične (engl. *plain*) neuronske mreže s 20 i 56 slojeva. Dublja mreža pokazuje veću pogrešku treniranja, a posljedično i veću pogrešku testiranja. Izvor: He, K., Zhang, X., Ren, S. i Sun, J. (2015).

Kako bi riješili te probleme, ResNet uvodi koncept rezidualnog učenja, gdje se u svakom bloku uči samo rezidual u odnosu na ulaz, dok se identitet propagira putem skip veza.

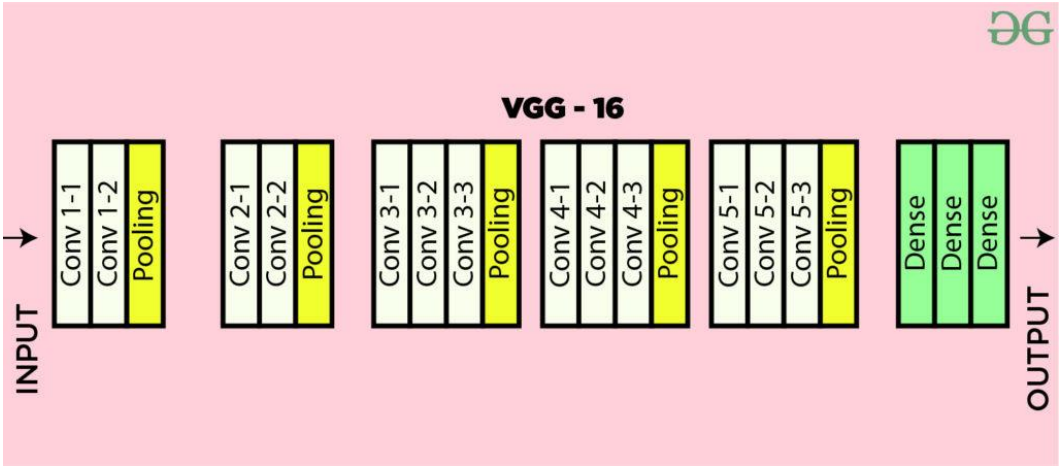
ResNet-18 sadrži 18 slojeva s težinama, organiziranih u 8 rezidualnih blokova. Rezidualni blok sastoji se od dvaju konvolucijskih slojeva s pripadnim Batch Normalization i ReLu aktivacijama te skip vezom koja dodaje ulaz bloka njegovom izlazu.



Slika 3.3: Shematski prikaz osnovnog residualnog bloka u ResNet arhitekturi. Ulazni signal *x* prolazi kroz dvije težinske (konvolucijske) slojevite transformacije s ReLu aktivacijom, čime se dobiva rezidualna funkcija *F(x)*. Rezultat se zbraja s izvornim ulazom putem identitetne (skip) veze, a dobiveni izlaz *F(x) + x* prolazi kroz završnu ReLu aktivaciju. *Izvor: Roboflow Blog – "What Is ResNet-18?" (2025).*

3.3 VGG-16

VGG-16 model je konvolucijske neuronske mreže koji se sastoji od 16 slojeva: 13 konvolucijskih slojeva te 3 potpuno povezana sloja. Glavna je ideja VGG-16 arhitekture koristiti više uzastopnih konvolucijskih slojeva s malim filtrima dimenzije 3x3, umjesto manjeg broja slojeva s većim filtrima. Takav pristup omogućuje veću dubinu mreže i veći broj nelinearnosti uz relativno jednostavan i uniforman dizajn. Iako je arhitektura VGG-16 poprilično jednostavna, ona sadrži puno parametara (oko 138 milijuna), što dovodi do velikih memorijskog zahtjeva, ali i sporijeg treniranja. Također, za razliku od ResNet-18 arhitekture, VGG-16 arhitektura ne koristi skip veze, zbog čega je osjetljivija na problem eksplodirajućeg gradijenta.

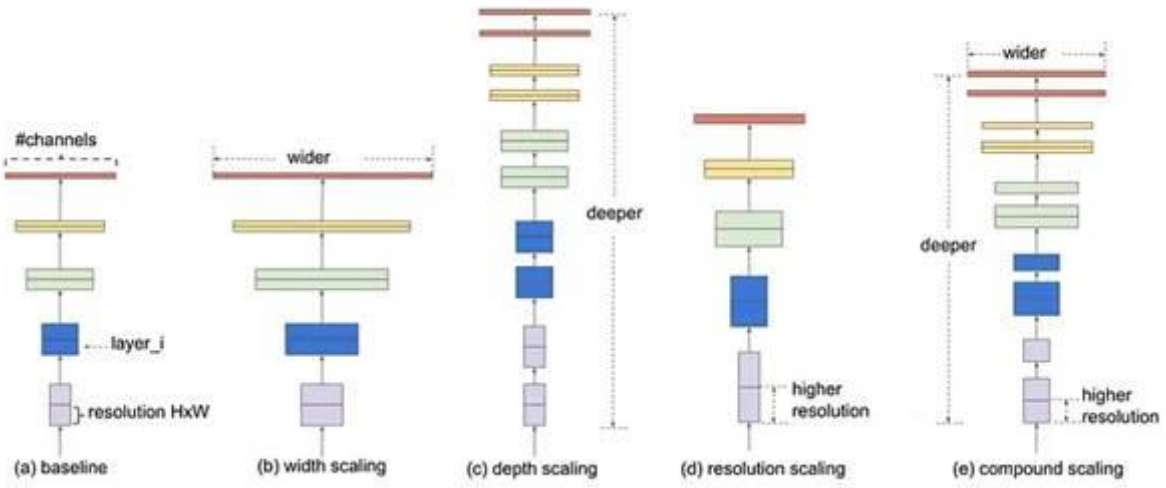


Slika 3.4: Shematski prikaz arhitekture VGG-16. Mreža se sastoji od pet konvolucijskih blokova s malim 3x3 filtrima, pri čemu svaki blok završava operacijom sažimanja (max pooling). Nakon konvolucijskog dijela slijede tri potpuno povezana (dense) sloja koji služe za konačnu klasifikaciju. Izvor: GeeksforGeeks, "VGG-16 CNN Model"

3.4 EfficientNetB0

Kod neuronskih mreža poznato je kako povećanjem dubine i širine same arhitekture model postaje kvalitetniji i precizniji. Međutim nije jasna metoda kako efektivno proširiti model za najbolje rezultate, stoga je EfficientNet obitelj neuronskih mreža bila dizajnirana da pronađe najbolji odnos između točnosti modela i efikasnosti izvršavanja.

Autori EfficientNet-a su istraživali sve moguće varijante. Razmatrali su dodavanje više neurona u postojećim slojevima, povećavanje broja slojeva te korištenje slika s većom rezolucijom. Testiranje svih mogućnosti otkrili su kako je najefikasnije sva tri parametra istovremeno povećati te stvoriti balans između njih. Ta metoda širenja mreže se naziva „Compound Scaling".



Slika 3.5: Prikaz različitih varijanti skaliranja modela. S lijeva na desno: a) Osnovni model, b) Skaliranje širine, c) Skaliranje dubine, d) Skaliranje rezolucije slike, e) Compound skaliranje. Izvor: GeeksforGeeks, "Efficientnet Architecture"

Prije objašnjavanja skaliranja, vrijedi pojasniti termin FLOPs. „Floating-Point Operations Per Second" je termin kojim prikazujemo koliko puno „floating-point" operacija možemo obaviti u sekundi. Ovaj termin možemo izračunati i za naše mreže, odnosno odrediti koliko su one zahtjevne za treniranje.

Idući izrazi prikazuju odnos koji su prisutni prilikom skaliranja modela.

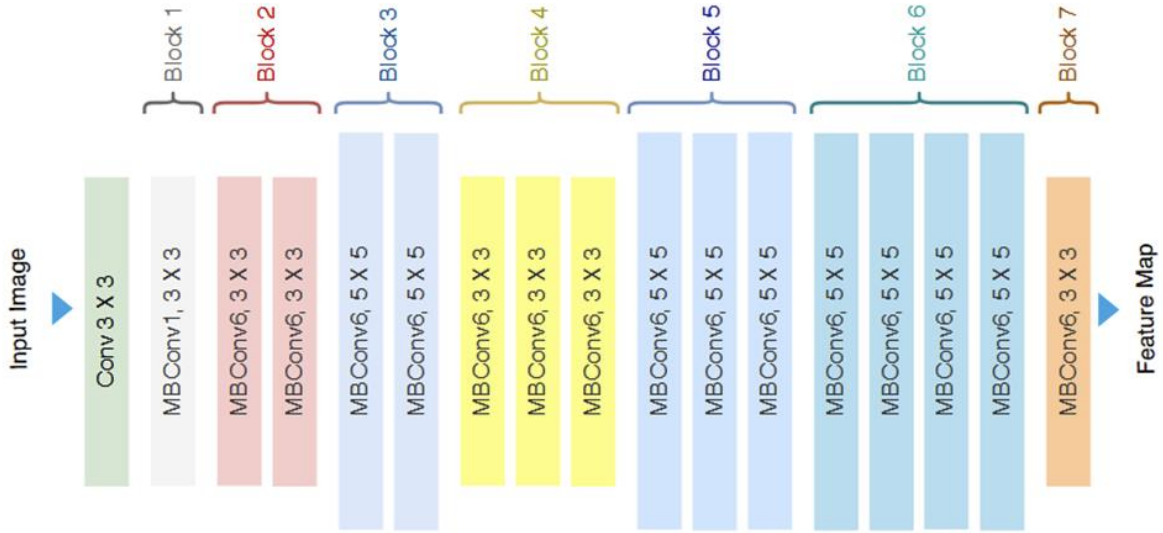
$$dubina: d = \alpha^{\theta}$$
$$širina: w = \beta^{\theta}$$
$$rezolucija: r = \gamma^{\theta}$$
$$FLOPS \propto d \cdot w^2 \cdot r^2 = (\alpha \cdot \beta^2 \cdot \gamma^2)^{\theta}$$
$$2 \approx \alpha \cdot \beta^2 \cdot \gamma^2$$
$$FLOPS \propto (2)^{\theta}$$
$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

Napomena: d, w, r su za EfficientNetB0 jednaki 1 te $\theta = 0$.
Razlog za to je jer se ostali modeli skaliraju s obzirom na broj i širinu slojeva te kvalitetu slika upravo tog modela.

Ideja je da napravimo osnovnu verziju arhitekture, u našem slučaju EfficientNetB0. Zatim odabiremo parametre α, β, γ koji će nam služiti za skaliranje duljine, širine modela te rezolucije ulaznih slika. Postoje preporučene vrijednosti za navedene parametre, ali nisu nužne. Nadalje kako bi skalirali ovaj model, jedino što trebamo je povećati vrijednost θ za cijeli broj kako bi nam se model ujednačeno povećao. Upravo skaliranjem tog parametra nastaju mreže

EfficientNetB1 ($\theta = 1$), sve do EfficientNetB7 ($\theta = 7$). Jednakost koja je navedena u šestom redu nam objašnjava kako za svako povećanje parametra za +1 (uz preduvjet u 5. redu), vrijednost FLOPs postaje dvostruko veća. Odnosno kompleksnost izvođenja postaje dvostruko veća.

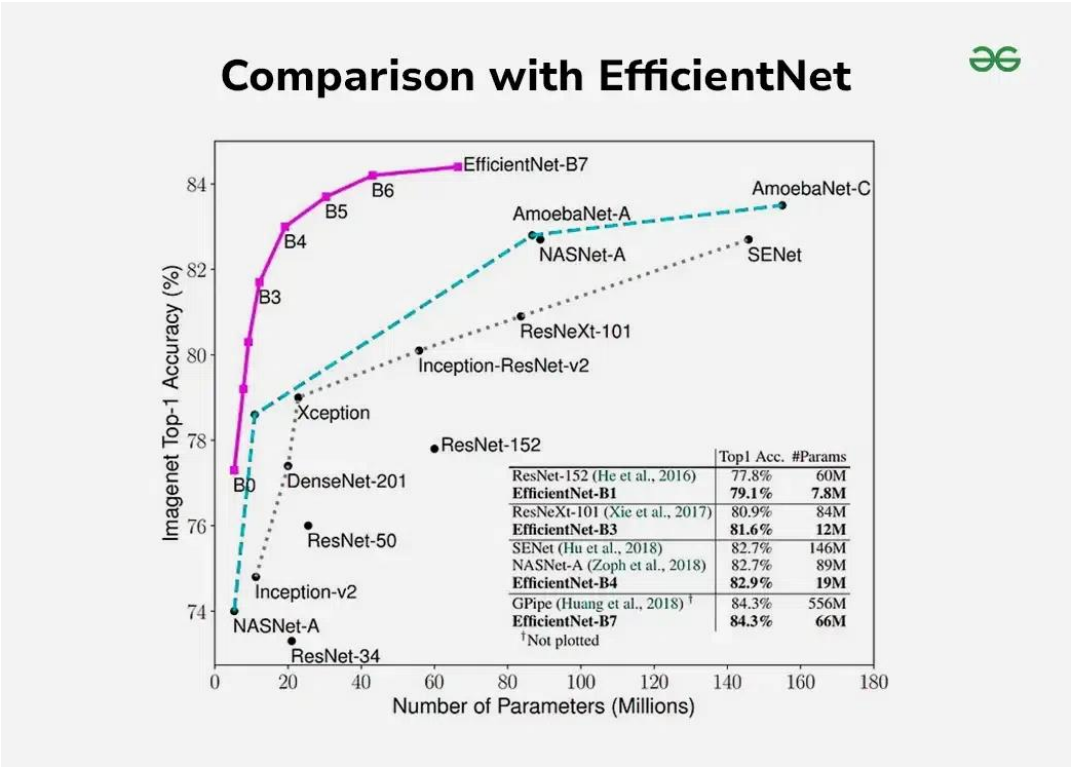
Kao što je prije navedeno za osnovni model EfficientNet obitelji mreža, bila je dizajnirana EfficientNetB0 arhitektura. Mreža se sastoji od 7 blokova koji u sebi imaju MBConv (Mobile Inverted Bottleneck) slojeve koji će kasnije biti objašnjeni u modelu EfficientNetB1.



Slika 3.6: Shematski prikaz arhitekture EfficientNetB0. Mreža se sastoji od ulaza s konvolucijskom matricom 3x3 uz normalizaciju i aktivacijsku funkciju, zatim sedam blokova koji se sastoje od MBConv slojeva. Model završava s konvolucijskom matricom 1x1 te linearnim slojem. Izvor: T. Ahmed, N.H.N. Sabab "Classification and understanding of cloud structures via satellite images with EfficientUNet"

3.5 EfficientNet-B1

EfficientNet-B1 je konvolucijska neuronska mreža iz obitelji EfficientNet, predstavljene u radu *"EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks"* (Tan i Le, 2019). Model je dizajniran s ciljem postizanja visoke točnosti uz minimalan broj parametara i računalnih operacija, korištenjem sustavnog pristupa skaliranju arhitekture.



Slika 3.7: Usporedba točnosti Top-1 na skupu ImageNet u odnosu na broj parametara za različite konvolucijske neuronske mreže. Prikazana je EfficientNet obitelj (B0–B7), koja postiže višu točnost uz znatno manji broj parametara u usporedbi s klasičnim arhitekturama poput ResNet-a, DenseNet-a, Inception-a i NASNet-a. Izvor: GeeksforGeeks, "EfficientNet Architecture"

EfficientNet-B1 temelji se na osnovnom modelu EfficientNet-B0, pri čemu se kapacitet mreže povećava primjenom compound scalinga, koji istovremeno i uravnoteženo skalira dubinu, širinu i ulaznu rezoluciju mreže. U odnosu na B0, EfficientNet-B1 koristi veću ulaznu rezoluciju i veći broj kanala, što rezultira poboljšanim performansama uz relativno mali porast računalnog troška.

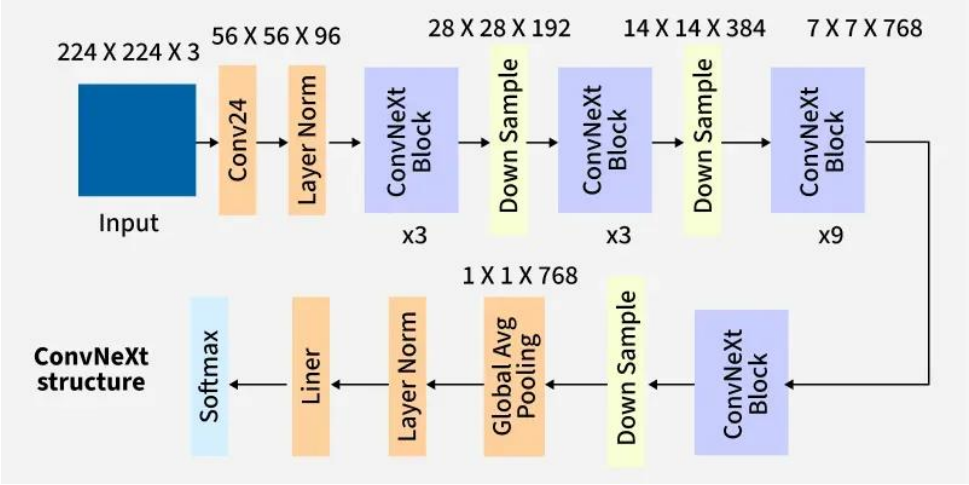
Osnovni građevni elementi EfficientNet-B1 su MBConv (Mobile Inverted Bottleneck) blokovi, koji kombiniraju ekspanzijsku 1x1 konvoluciju, depthwise separable konvoluciju i projekcijsku 1x1 konvoluciju. Dodatno, MBConv blokovi uključuju Squeeze-and-Excitation (SE) mehanizam, koji adaptivno re-skalira kanale na temelju globalnog konteksta značajki. U slučajevima kada se dimenzije ulaza i izlaza podudaraju, primjenjuju se rezidualne (skip) veze, čime se poboljšava stabilnost treniranja i propagacija gradijenata.

Zahvaljujući kombinaciji učinkovitih konvolucijskih blokova i uravnoteženog skaliranja, EfficientNet-B1 postiže povoljan omjer između točnosti i računalne složenosti te se često koristi u okruženjima s ograničenim računalnim resursima.

3.6 ConvNeXt

ConvNeXt je moderna konvolucijska neuronska mreža, koja umjesto uvođenja novih operacija zadržava klasične konvolucije, ali primjenjuje suvremene dizajnerske principe inspirirane Transformer arhitekturama.

Osnovni građevni element ConvNeXt-a je ConvNeXt blok, koji predstavlja moderniziranu verziju ResNet bloka. Svaki blok započinje 7x7 depthwise konvolucijom, nakon čega slijedi Layer Normalization, koja zamjenjuje Batch Normalization, te dvije 1x1 konvolucije. Kao nelinearna aktivacija koristi se GELU, a izlaz bloka se, uz odgovarajuće dimenzije, zbraja s ulazom putem rezidualne (skip) veze. Zahvaljujući velikim depthwise kernelima, LayerNorm normalizaciji, pre-norm dizajnu i pažljivo odabranim trening strategijama, ConvNeXt postiže visoku točnost uz zadržavanje konvolucijske strukture modela.



Slika 3.8: Struktura ConvNeXt arhitekture. Ulazna slika dimenzija 224×224×3 prolazi kroz početni stem sloj (4x4 konvolucija i Layer Normalization), nakon čega slijede četiri hijerarhijska stage-a sastavljena od ConvNeXt blokova. U svakom stage-u prostorna rezolucija se smanjuje (downsampling), dok se broj kanala povećava. Na kraju mreže primjenjuje se globalno prosječno sažimanje (global average pooling), nakon čega slijede normalizacija i linearni klasifikator s funkcijom Softmax. Izvor: GeeksforGeeks, "ConvNeXt Architecture".

4 Opis metoda klasifikacije neuronskih mreža

4.1 Treniranje mreže iz nule

Cilj strojnog učenja je minimizirati očekivanu pogrešku generalizacije, no s obzirom da očekivanu pogrešku ne znamo, navedeni problem se ne može svesti na optimizacijski problem, već uz pomoć skupa uzoraka za učenje procjenimo pravu distribuciju na temelju empirijske distribucije. Nadamo se da će smanjenjem empirijskog rizika padati i očekivani gubitak uzet po svim uzorcima iz izvorne distribucije skupa uzoraka za učenje. Algoritmi za učenje koje smo koristili su SGD (Stohastic Gradient Descent) i Adam (Adaptive Moments).

Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k.
Require: Initial parameter $\boldsymbol{\theta}$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\boldsymbol{g}}$
  end while

Slika 4.1: Algoritam Stohastic Gradient Descent

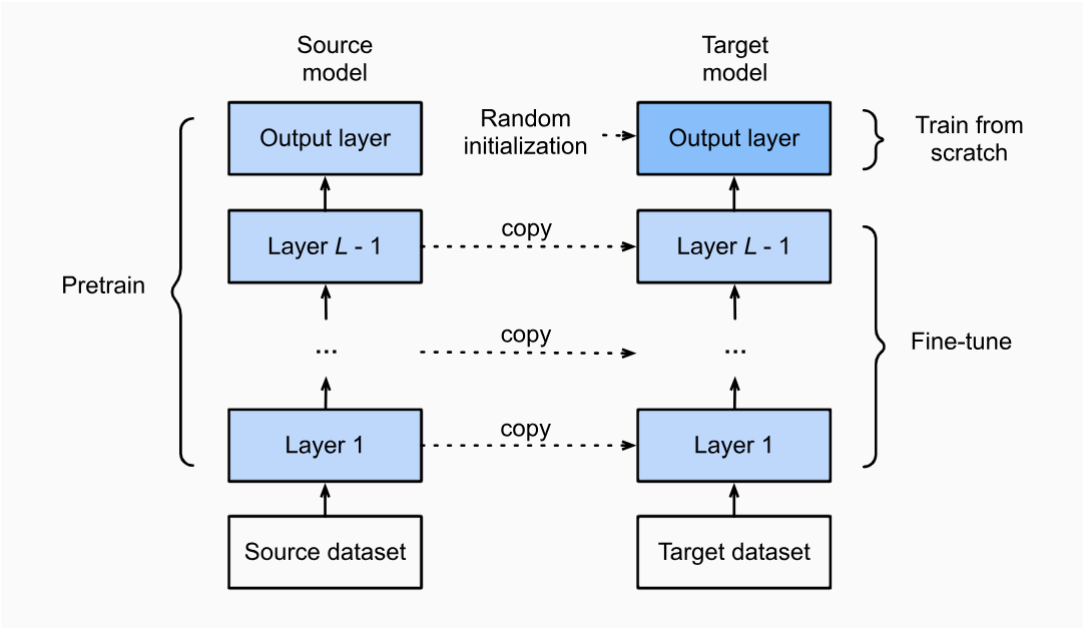
Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)
Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$. (Suggested defaults: 0.9 and 0.999 respectively)
Require: Small constant δ used for numerical stabilization. (Suggested default: 10^{-8})
Require: Initial parameters $\boldsymbol{\theta}$
  Initialize 1st and 2nd moment variables $\boldsymbol{s} = \boldsymbol{0}$, $\boldsymbol{r} = \boldsymbol{0}$
  Initialize time step $t = 0$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    $t \leftarrow t + 1$
    Update biased first moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1) \boldsymbol{g}$
    Update biased second moment estimate: $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2) \boldsymbol{g} \odot \boldsymbol{g}$
    Correct bias in first moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}$
    Correct bias in second moment: $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t}$
    Compute update: $\Delta \boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (operations applied element-wise)
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$
  end while

Slika 4.2: Algoritam Adaptive Moments

4.2 Fine-tuning mreže

Kada želimo suziti područje primjene modela, odnosno želimo ga specijalizirati koristimo se finim podešavanjem (fine-tuning). Fino podešavanje postižemo tako što predtreniramo izvorni model na izvornom skupu podataka te dupliciramo sve slojeve tog modela osim izlaznog sloja kojeg sami dodajemo. S obzirom da svi slojevi osim izlaznog imaju parametre izvornog modela, kažemo da se oni fino podešavaju dok se izlazni sloj trenira iz nule.



Slika 4.3: Postupak finog podešavanja

4.3 Treniranje klasifikacijske glave

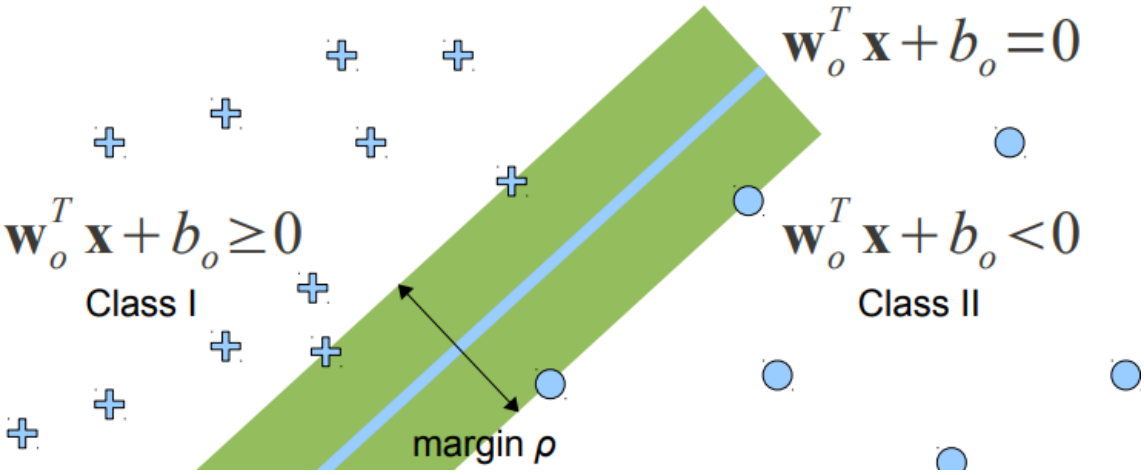
Klasifikacijska glava se odnosi na zadnji sloj neuronske mreže koji prima naučene podatke iz prijašnjih slojeva i koristi ih za pretpostavke. Prilikom treniranja mreže svi se slojevi zamrznu, ne mijenjamo težine modela tijekom backpropagation algoritma, osim posljednjeg sloja modela. Zamrzavanjem slojeva osiguravamo da model ne zaboravi temeljne obrasce poput rubova, tekstura i oblika.

4.4 Linear probe

Linear probes su jednostavni linearni klasifikatori (npr. softmax regresija ili logistička regresija) koje dodajemo na unutarnje slojeve već treniranog neuronskog modela kako bismo istražili što je model naučio na tim slojevima. Prilikom klasifikacije linear probe-ovima, težine glavnog modela se ne mijenjaju, tj. glavni model je zamrznut. Linear probe nam kaže koliko je linearna informacija dostupna u aktivacijama sloja. Ako linearni model može dobro klasificirati zadatak samo na tim aktivacijama, to znači da su značajke dobro „razdvojene“ u tom prostoru.

4.5 Stroj s potpornim vektorima (SVM)

SVM je binarni klasifikacijski algoritam koji nastoji pronaći hiperravninu u visokodimenzionalnom prostoru značajki koja razdvaja podatkovne točke različitih klasa s maksimalnom marginom. Margina predstavlja udaljenost hiperravnine i najbliže podatkovne točke svake klase. Prilikom treniranja modela nastojimo pronaći optimalne težine koje definiraju hiperravninu rješavanjem optimizacijskog problema.



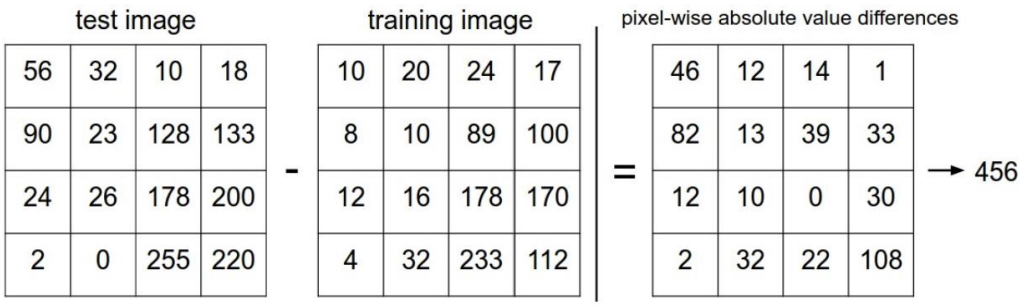
Slika 4.4: Vizualizacija klasifikacije primjenom SVM-a. Prikaz hiperravnine, margine i klasa nastalih klasifikacijom.

Nakon pronalaska hiperravnine, uz pomoć SVM-a možemo klasificirati podatkovne točke. Prilikom klasificiranja nove podatkovne točke SVM izračunava točkasti umnožak između vektora težine i vektora obilježja podatkovne točke. Ovisno o predznaku dobivenog umnoška svrstavamo novu podatkovnu točku na odgovarajuću stranu hiperravnine te iznos samog umnoška nam daje informaciju o udaljenosti podatkovne točke od hiperravnine.

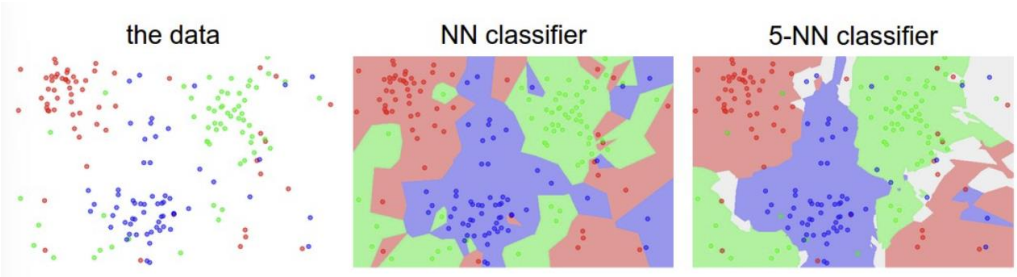
4.6 k nearest neighbours

Jedan od algoritama za klasifikaciju je Nearest Neighbor (NN) klasifikator, koji se temelji na izračunu L1 ili L2 udaljenosti između slike koju želimo klasificirati i svih slika iz skupa za treniranje. Preciznije, algoritam pronalazi sliku iz skupa za učenje koja ima najmanju udaljenost u odnosu na promatranu sliku te joj pridružuje istu oznaku klase. Budući da je navedena metoda neefikasna te često daje lošije rezultate, uvodi se k-Nearest Neighbor (kNN) klasifikator. Umjesto da odluku temelji na samo jednom najbližem primjeru, kNN razmatra k

po udaljenosti najbližih slika iz skupa za učenje te klasu određuje glasanjem većine među njima, čime se postiže preciznija klasifikacija.



Slika 4.5: Izračun L1 udaljenosti dvaju slika



Slika 4.6: Vizualizacija klasa nastalih primjenom Nearest Neighbour i k Nearest Neighbour algoritama

5 Dobiveni rezultati

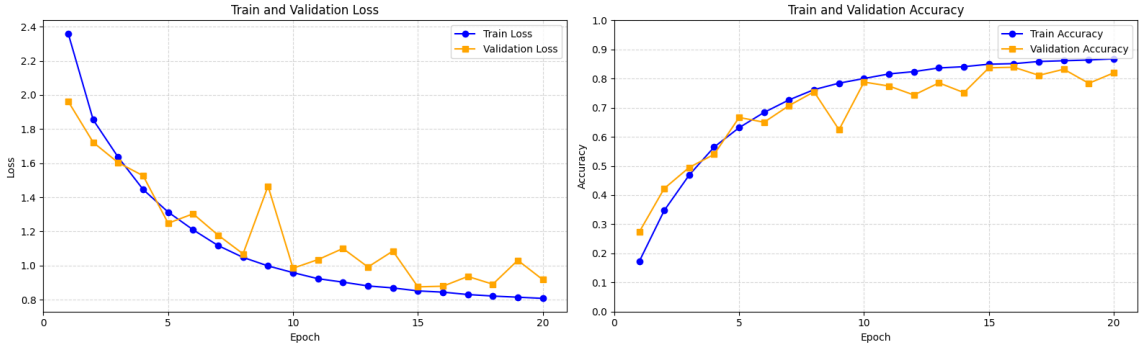
U nastavku su navedeni rezultati dobiveni primjenom prethodno objašnjenih klasifikacijskih metoda na 3 dataseta: Cifar10, Cifar100 i Imagenette.

5.1 Treniranje iz nule

Testirali smo na različitim konfiguracijama poput različitih faktora učenja optimizatora, veličine batcha i augmentacije. Prikazani rezultati su rezultati koji koriste najbolju konfiguraciju.

	Train_loss	Train_ac	Val_loss	Val_acc	Test_los	Test_ac
MKM	0.2801	0.9026	0.6990	0.7873	0.7334	0.7831
ResNet18	0.5011	1.0000	0.6196	0.9556	0.6421	0.9467
EfficientNetB0	0.1382	0.9520	0.3190	0.8991	*0.3494*	*0.8911*

Treniranje iz nule na datasetu Cifar10



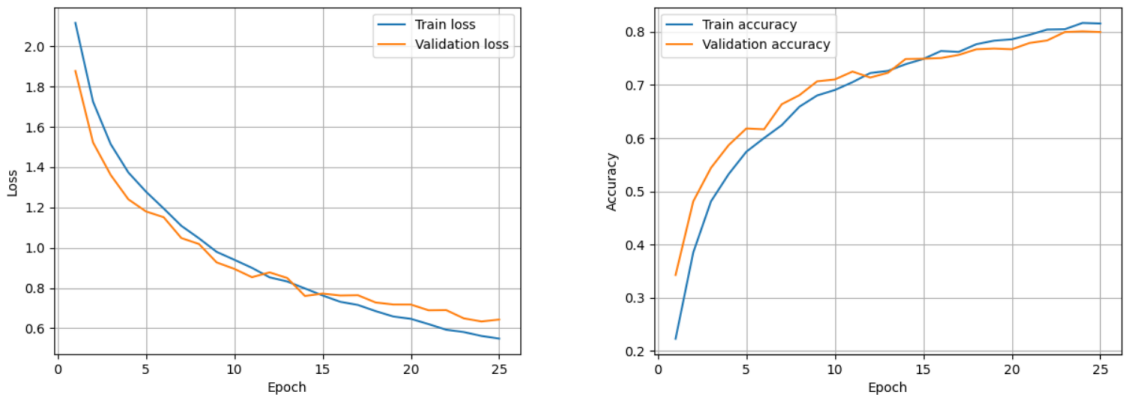
Graf najboljih rezultata – za ResNet18

	Train_loss	Train_acc	Val_los	Val_ac	Test_loss	Test_ac
MKM	1.4422	0.6035	1.9685	0.5100	1. 9378	0. 5169
ResNet18	0.7549	0.7791	1.6570	0.5713	1.6534	0.5791
EfficientNetB0	0.8813	0.7309	1.4783	0.6126	1.4230	0.6145

Treniranje iz nule na datasetu Cifar100

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
MKM	1.0308	0.6721	1.1797	0.6181	1.2654	0.6245
ResNet18	0.5571	0.8152	0.9545	0.7049	0.7356	0. 7694
EfficientNetB0	0.5487	0.8158	0.6433	0.7997	0.6433	0.7997

Treniranje iz nule na datasetu Imagenette



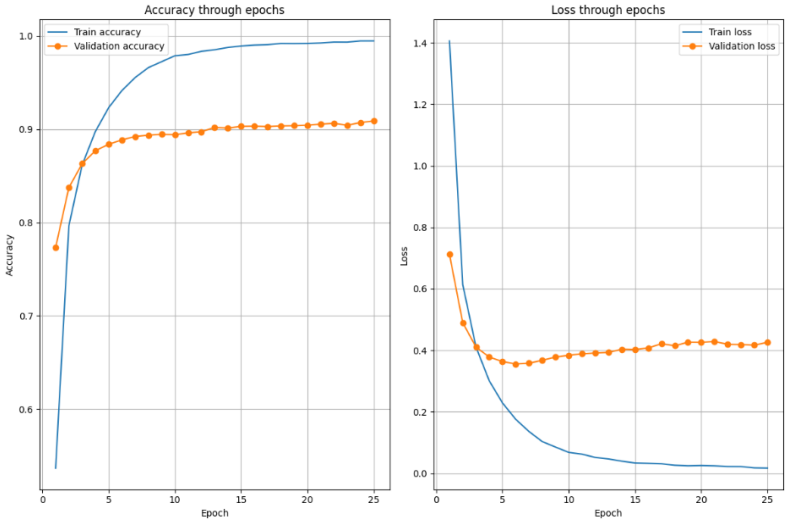
Graf najboljih rezultata – za EfficientNetB0

5.2 Fine-tuning

Koristimo predtrenirane težine ImageNet1k.

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0.5295	0.8156	0.5327	0.8124	0.5370	0.8110
EfficientNetB0	0.0205	0.9934	0.4504	0.9046	0.4775	0.8997

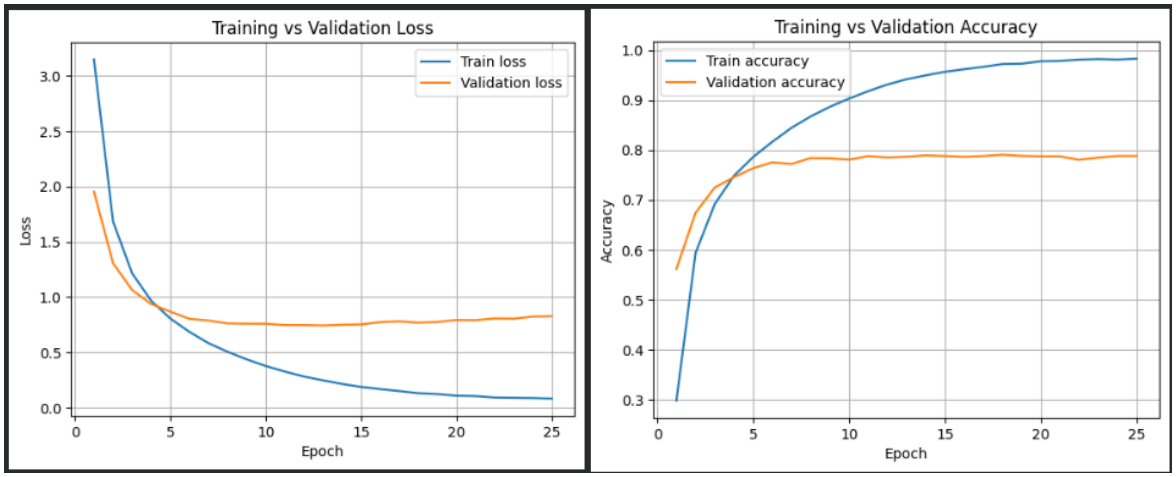
Fino podešavanje na datasetu Cifar10



Graf najboljih rezultata – za EfficientNetB0

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0.0849	0.9829	0.8289	0.7879	0.8362	0.7852
EfficientNetB0	1.2979	0.6431	1.3310	0.6359	1.3448	0.6332

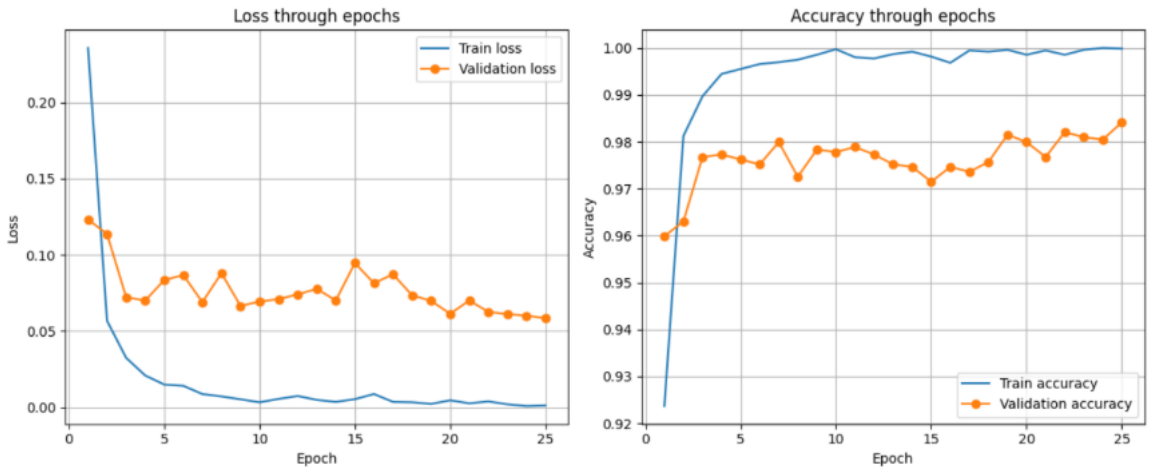
Fino podešavanje na datasetu Cifar100



Graf najboljih rezultata – za ResNet-18

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0.0017	0.9999	0.0655	0.9836	0.0717	0.9791
EfficientNetB0	0.0091	0.9984	0.0656	0.9826	0.0656	0.9801

Fino podešavanje na datasetu Imagenette



Graf najboljih rezultata – za ResNet-18

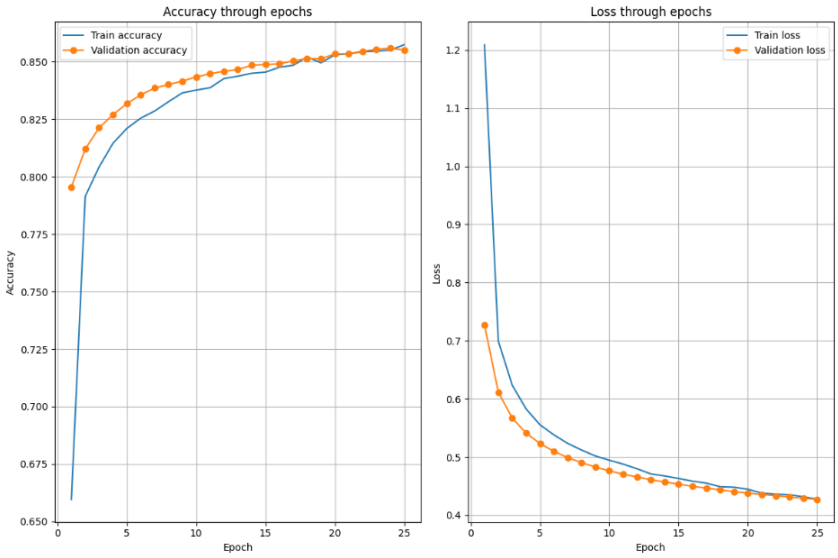
5.3 Treniranje klasifikacijske glave

Koristimo predtrenirane težine ImageNet1k.

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	1.6413	0.4264	1.6825	0.4189	1.6943	0.4168

Vgg16	0.7028	0.7522	0.6627	0.7666	0.6880	0.7567
EfficientNetB0	1.2211	0.5849	1.1676	0.6056	1.2067	0.5986
EfficientNetB1	1.2810	0.5704	1.2293	0.5886	1.2500	0.5769
ConvNext	0.4275	0.8573	0.4271	0.8550	0.4467	0.8503

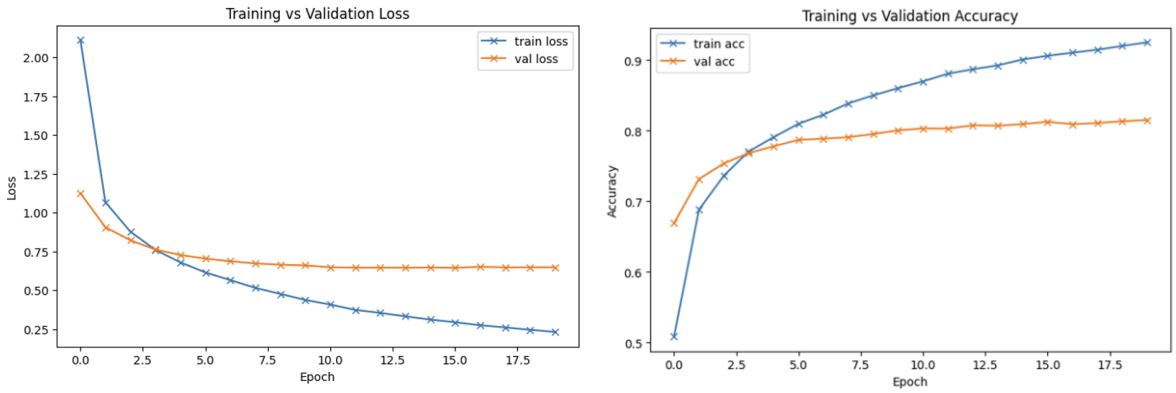
Treniranje klasifikacijske glave na datasetu Cifar10



Graf najboljih rezultata – za ConvNeXt

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0.6667	0.7888	1.3215	0.6566	1.2845	0.6588
Vgg16	0.2834	0.9466	1.0767	0.6881	1.0875	0.6884
EfficientNetB0	1.3263	0.6318	1.3099	0.6362	1.3085	0.6383
EfficientNetB1	0.2306	0.9252	0.6470	0.8153	0.6347	0.8200
ConvNext	-	0.5440	-	0.6106	-	0.6118

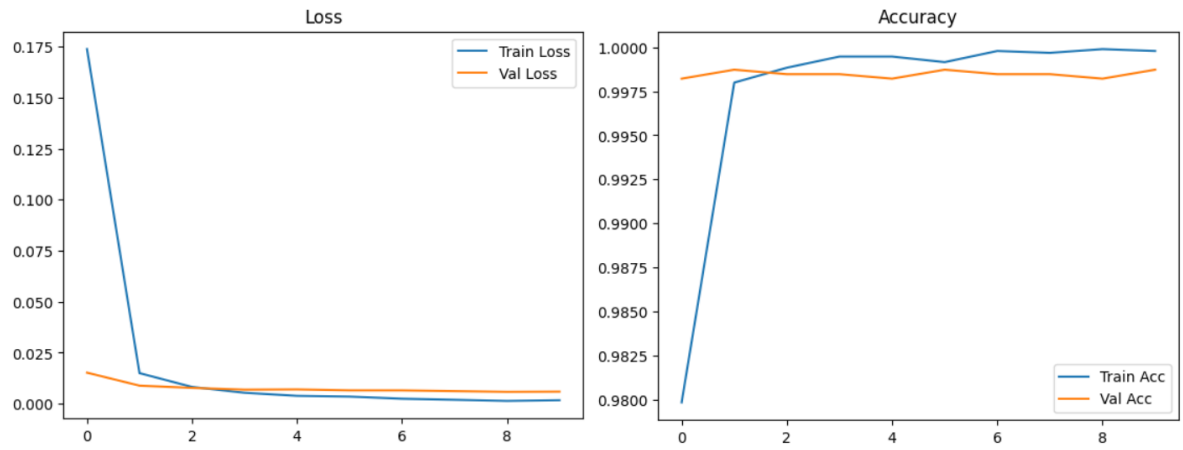
Treniranje klasifikacijske glave na datasetu Cifar100



Graf najboljih rezultata – za EfficientNetB1

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0.0238	0.9937	0.1696	0.9747	0.1321	0.9755
Vgg16	0.0253	0.9931	0.1955	0.9688	0.2379	0.9615
EfficientNetB0	0.0402	0.9856	0.0699	0.9805	0.0806	0.9763
EfficientNetB1	0.0345	0.9888	0.0478	0.9847	0.0605	0.9837
ConvNext	0.0017	0.9996	0.0068	0.9979	0.0105	0.9982

Treniranje klasifikacijske glave na datasetu Imagenette



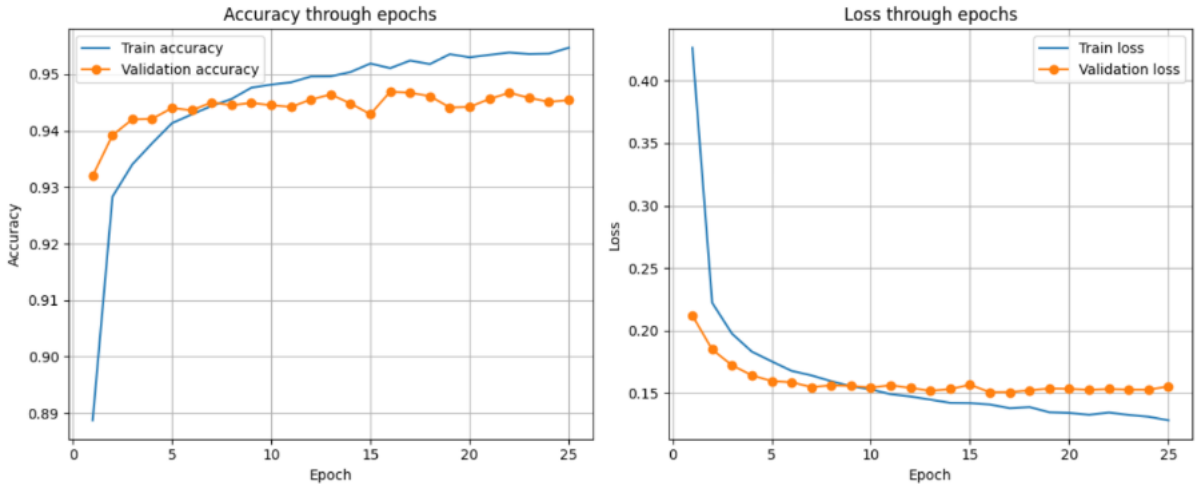
Graf najboljih rezultata – za ConvNext

5.4 Linear probe

Koristimo predtrenirane težine ImageNet1k.

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0.7326	0.6732	0.6553	0.7599	0.6598	0.7562
Vgg16	0.5402	0.8147	0.4366	0.8430	0.4620	0.8400
EfficientNetB0	0.6060	0.7929	0.5223	0.8198	0.5344	0.8142
EfficientNetB1	0.5479	0.8079	0.4971	0.8258	0.4987	0.8322
ConvNext	0.1282	0.9547	0.1554	0.9454	0.1740	0.9411

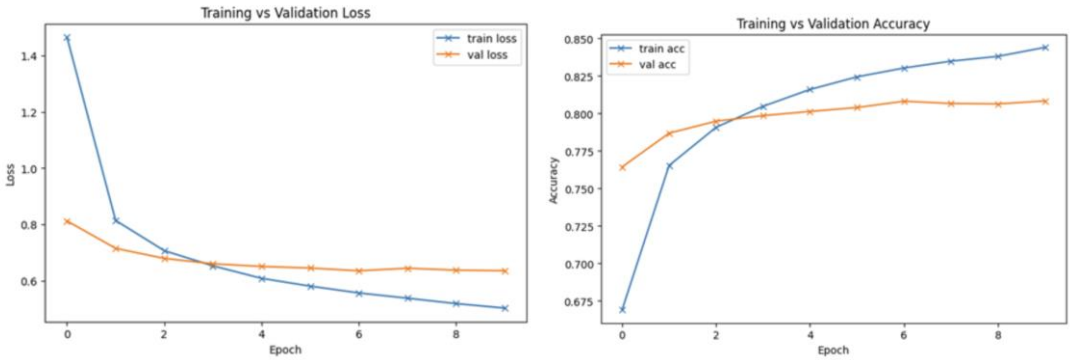
Linear probe na datasetu Cifar10



Graf najboljih rezultata – za ConvNext

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0. 6667	0. 7888	1. 3215	0. 6566	1. 2845	0. 6588
Vgg16	1.4950	0.7436	1.665	0.6187	1.675	0.6150
EfficientNetB0	1.3823	0.6229	2.2605	0.4338	1.6632	0.4379
EfficientNetB1	1.5130	0.5913	1.6516	0.5714	1.6556	0.5671
ConvNext	0.5022	0.8441	0.6352	0.8084	0.6377	0.8113

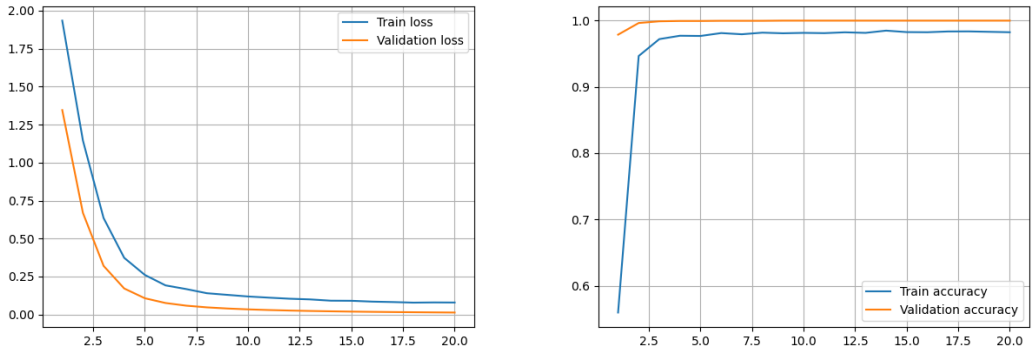
Linear probe na datasetu Cifar100



Graf najboljih rezultata – za ConvNext

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	0.0657	0.9798	0.1209	0.9694	0. 1192	0. 9605
Vgg16	0.0657	0.9834	0.0657	0.9834	0.1613	0.9542
EfficientNetB0	0.1690	0.9623	0.0870	0.9855	0.0870	0.9855
EfficientNetB1	0.2495	0.9630	0.1321	0.9952	0.1321	0.9952
ConvNext	0.0788	0.9823	0.0128	0.9997	0.0128	0.9997

Linear probe na datasetu Imagenette



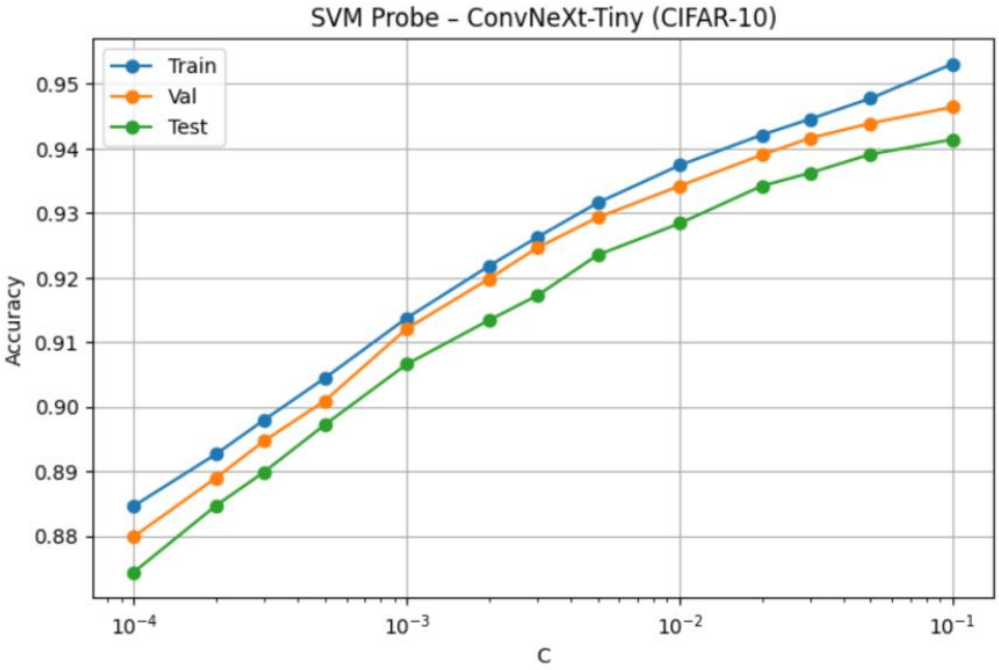
Graf najboljih rezultata – za ConvNext

5.5 SVM

Koristimo predtrenirane težine ImageNet1k.

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	-	0.8654	-	0.8541	-	0.845
Vgg16	-	0.8510	-	0.8346	-	0. 8075
EfficientNetB0	-	0.9297	-	0.9151	-	0. 9116
EfficientNetB1	-	0.9295	-	0.9169	-	0. 9126
ConvNext	-	0.9531	-	0.9464	-	0. 9403

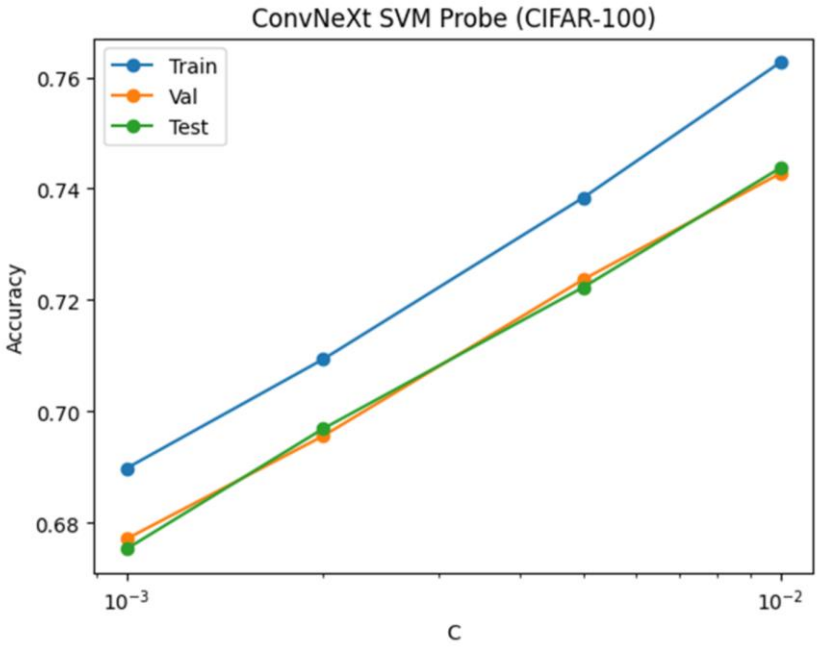
Upotreba SVM-a na datasetu Cifar10



Graf najboljih rezultata – za ConvNext

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	-	0.7223	-	0.6605	-	0.6641
Vgg16	-	0.5854	-	0.5566	-	0.5469
EfficientNetB0	-	0.7597	-	0.7159	-	0.7107
EfficientNetB1	-	0.9617	-	0.7225	-	0.7230
ConvNext	-	0.7627	-	0.7427	-	0.7437

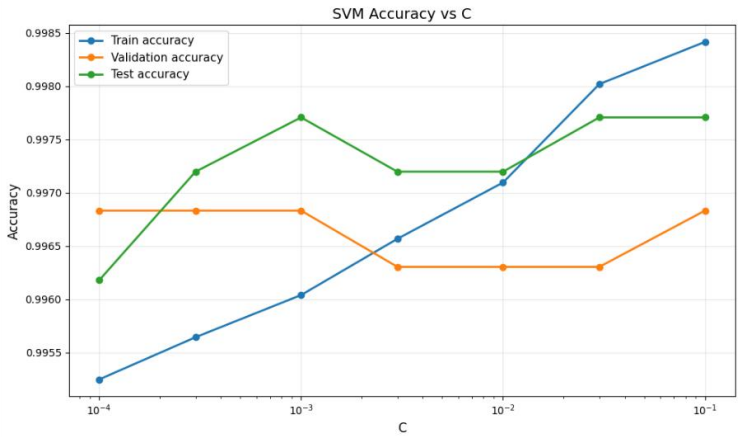
Upotreba SVM-a na datasetu Cifar100



Graf najboljih rezultata – za ConvNext

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	-	0.9938	-	0.9741	-	0. 9654
Vgg16	-	0.9768	-	0.9662	-	0.9684
EfficientNetB0	-	0.9888	-	0.9820	-	0.9801
EfficientNetB1	-	0.9901	-	0.9847	-	0.9819
ConvNext	-	0.9984	-	0.9968	-	0.9977

Upotreba SVM-a na datasetu Imagenette



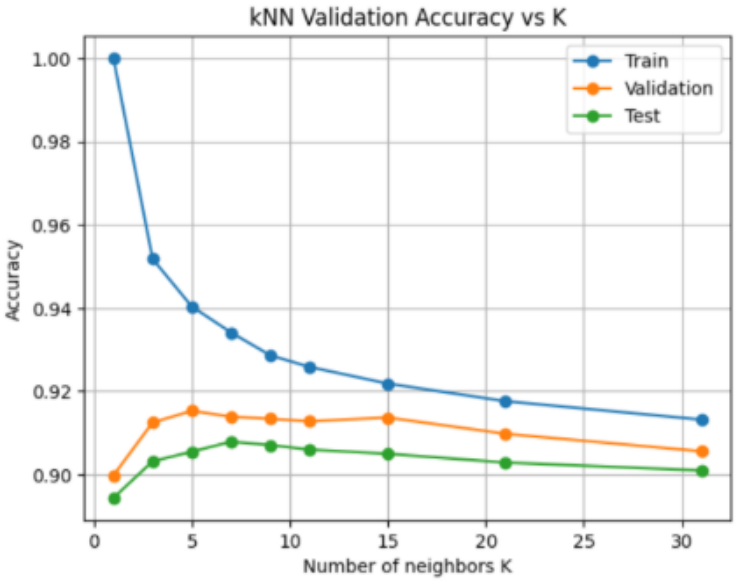
Graf najboljih rezultata – za ConvNext

5.6 k nearest neighbours

Koristimo predtrenirane težine ImageNet1k.

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	-	0.8859	-	0.8573	-	0.8464
Vgg16	-	0.8301	-	0.7990	-	0.7882
EfficientNetB0	-	0.9152	-	0.8970	-	0.8934
EfficientNetB1	-	0.9068	-	0.8983	-	0.8952
ConvNext	-	0.9404	-	0.9153	-	0.9055

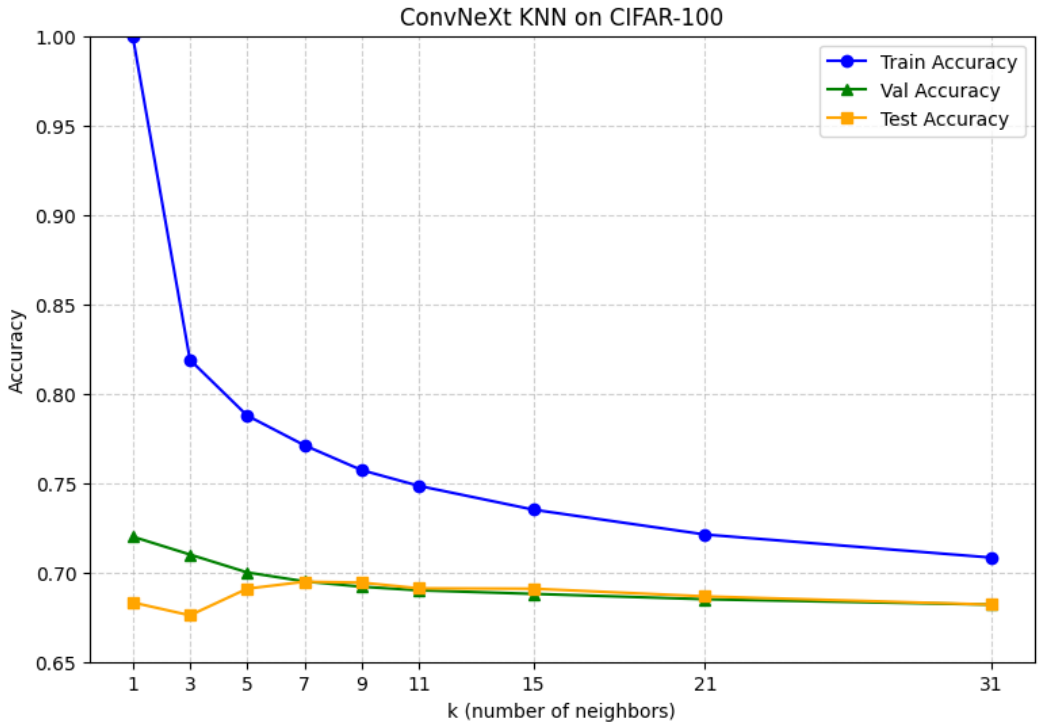
Upotreba k nearest neighbour na datasetu Cifar10



Graf najboljih rezultata – za ConvNext

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	-	0.6656	-	0.5889	-	0.5919
Vgg16	-	0.6206	-	0.5554	-	0.5561
EfficientNetB0	-	0.7290	-	0.6812	-	0.6776
EfficientNetB1	-	0.7857	-	0.6650	-	0.6674
ConvNext	-	0.7711	-	0.6990	-	0.6948

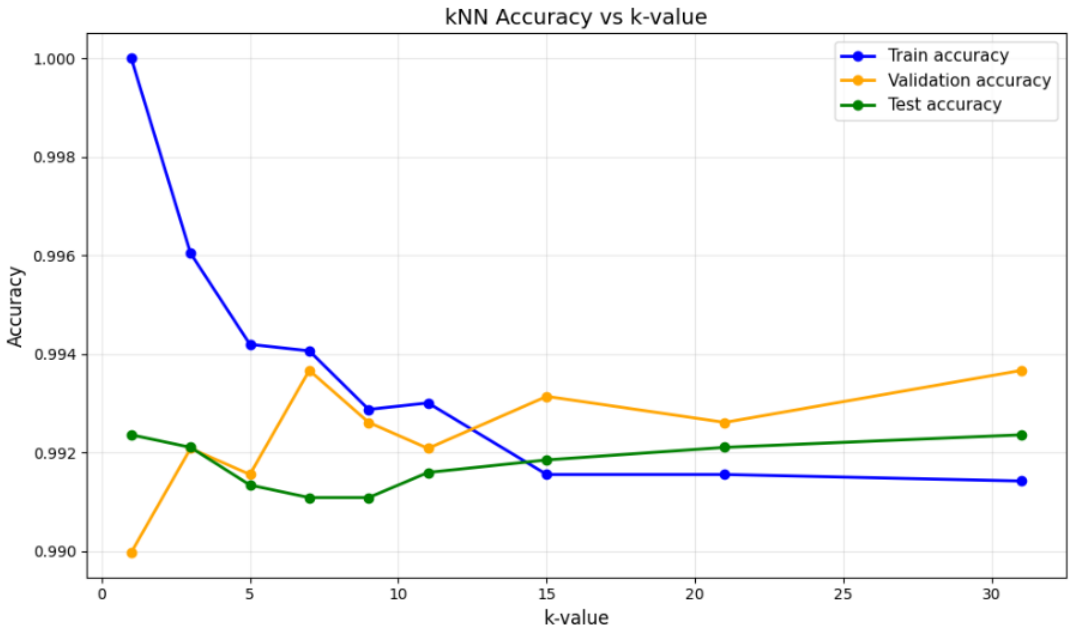
Upotreba k nearest neighbour na datasetu Cifar100



Graf najboljih rezultata – za ConvNext

	Train_loss	Train_acc	Val_loss	Val_acc	Test_loss	Test_acc
ResNet18	-	0.9791	-	0.9725	-	0.9702
Vgg16	-	0.9708	-	0.9646	-	0.9587
EfficientNetB0	-	0.9844	-	0.9757	-	0.9743
EfficientNetB1	-	0.9832	-	0.9826	-	0.9773
ConvNext	-	0.9941	-	0.9937	-	0.9911

Upotreba k nearest neighbour na datasetu Imagenette



Graf najboljih rezultata – za ConvNext

6. Zaključak

	Cifar10	Cifar100	Imagenette
MKM	0.7831 (od nule)	0.5169 (od nule)	0.6245 (od nule)
ResNet18	0.9467 (od nule)	0.7852 (fine-tuning)	0.9791 (fine-tuning)
Vgg16	0.840 (linear probe)	0.6884 (klas. glava)	0.9684(SVM)
EfficientNetB0	0.9116 (SVM)	0.7107 (SVM)	0.9801(fine-tuning i SVM isti rezultat)
EfficientNetB1	0.9126 (SVM)	0.8200 (klas. glava)	0.9819 (SVM)
ConvNext	0.9411 (linear probe)	0.8113 (linear probe)	0.9977 (SVM)

Prikaz najbolje postignutih točnosti na test skupu (Test Accuracy) za svaku mrežu i dataset.

U ovom projektu provedena je detaljna analiza i usporedba različitih metoda klasifikacije slika temeljenih na dubokim konvolucijskim neuronskim mrežama. Implementirane su različite vrste mreža: mala konvolucijska mreža, ResNet-18, VGG-16, EfficientNetB0, EfficientNetB1 i ConvNeXt, kao i više metoda klasifikacije, poput treniranja iz nule, finog podešavanja (fine tuning), treniranja klasifikacijske glave, linear probe metode, SVM-a i k-najbližih susjeda (k Nearest Neighbours). Eksperimente smo proveli nad skupovima podataka CIFAR10, CIFAR100 i Imagenette.

Posebno se istaknuo ConvNeXt koji je u većini eksperimenata ostvario najbolje rezultate, neovisno o metodi klasifikacije. Također, metode klasifikacije SVM i k-najbližih susjeda su ostvarile najvišu točnost na testnom skupu što znači da su se pokazale kao najučinkovitije metode klasifikacije.

Dobiveni rezultati nam potvrđuju da su odabir vrste mreže, metode klasifikacije i hiperparametara ključni u postizanju optimalnih rezultata u klasifikaciji slika.

7. Literatura

[1] K. He, X. Zhang, S. Ren i J. Sun, "Deep Residual Learning for Image Recognition", arXiv preprint arXiv:1512.03385, 2015. Dostupno na: https://arxiv.org/pdf/1512.03385

[2] Medium, "A Simple Convolutional Neural Network (CNN) Classifier Based on Real Images", Dostupno na: https://medium.com/@chenycy/a-simple-convolutional-neural-network-cnn-classifier-based-on-real-images-084110d52c18

[3] Roboflow, "What Is ResNet-18?", Roboflow Blog, 2025. Dostupno na: https://blog.roboflow.com/resnet-18/

[4] GeeksforGeeks, "VGG-16 CNN Model", Dostupno na: https://www.geeksforgeeks.org/computer-vision/vgg-16-cnn-model/

[5] GeeksforGeeks, "EfficientNet Architecture", Dostupno na: https://www.geeksforgeeks.org/computer-vision/efficientnet-architecture/

[6] T. Ahmed, N.H.N. Sabab, "Classification and understanding of cloud structures via satellite images with EfficientUNet", Dostupo na: https://arxiv.org/pdf/2009.12931

[5] Medium, "EfficientNet: Smarter, Not Just Bigger, Neural Networks", Dostupno na https://medium.com/@kdk199604/efficientnet-smarter-not-just-bigger-neural-networks-94db3e2f8699

[6] GeeksforGeeks, "Floating-Point Operations Per Second (FLOPS)", Dostupno na: https://www.geeksforgeeks.org/computer-organization-architecture/what-is-floating-point-operations-per-second-flops/

[7] GeeksforGeeks, "ConvNeXt Architecture", Dostupno na: https://www.geeksforgeeks.org/computer-vision/convnext/https://www.geeksforgeeks.org/computer-vision/convnext/

[8] Eitca, " Što je Support Vector Machine (SVM)?", Dostupno na: https://hr.eitca.org/umjetna-inteligencija/eitc-ai-mlp-strojno-u%C4%8Denje-s-pythonom/stroj-za-podr%C5%A1ku-vektora/SVM-parametri/%C5%A1to-je-potporni-vektorski-stroj-svm/?utm_source=chatgpt.com

[9] Medium "A Comprehensive Guide to Convolutional Neural Networks" Dostupno na: https://maryam-sikander.medium.com/a-comprehensive-guide-to-convolutional-neural-networks-ad850c578d45

[10] IBM What are convolutional neural networks? Dostupno na: https://www.ibm.com/think/topics/convolutional-neural-networks

[11] Deep Learning Book Chapter 9: Convolutional Networks Dostupno na: https://www.deeplearningbook.org/contents/convnets.html

[12] *"Research on image classification model based on deep convolution neural network"* (Xin & Wang, 2019) Dostupno na: https://link.springer.com/article/10.1186/s13640-019-0417-8

[13] *"Dive into Deep Learning"* (Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J. Smola) Dostupno na: https://d2l.ai/index.html

[14] *"scikit-learn"* Dostupno na: https://scikit-learn.org/stable/index.html

[15] *"CS231n Deep Learning for Computer Vision"* Dostupno na: https://cs231n.github.io/

[16] *"Neslužbene stranice predmeta Duboko učenje 1"* Dostupno na:

https://www.zemris.fer.hr/~ssegvic/du/index.shtml

[17] Medium "Neural Network Heads" Dostupno na: https://medium.com/@AILearningHub/neural-network-heads-16ce560c0b10

[18] Medium "Guide to Freezing Layers in PyTorch: Best Practices and Practical Examples" Dostupno na: https://medium.com/we-talk-data/guide-to-freezing-layers-in-pytorch-best-practices-and-practical-examples-8e644e7a9598